Experiment No. 1

DFS and BFS Traversal in Graphs

Aim

To implement Depth First Search (DFS) and Breadth First Search (BFS) algorithms using an undirected graph and develop a recursive algorithm to search all vertices of a graph or tree data structure.

Objectives

1. Understand the concept of graph traversal using DFS and BFS.
2. Implement DFS using recursion and BFS using a queue.
3. Explore traversal techniques for undirected graphs and trees.
4. Analyze the time and space complexity of both algorithms.
5. Apply traversal logic to real-world problems like pathfinding and connectivity.

Theory

What is a Graph?

A graph is a data structure consisting of **vertices (nodes)** and **edges (connections)**. It can be:

- **Undirected**: Edges have no direction.
- **Connected**: Every vertex is reachable from any other vertex.

Depth First Search (DFS)

DFS explores as far as possible along each branch before backtracking. It uses:

- **Recursion** or a **stack**
- Marks visited nodes to avoid cycles

Recursive DFS Pseudocode:

python

```
def dfs(graph, vertex, visited):
    visited.add(vertex)
    for neighbor in graph[vertex]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

Breadth First Search (BFS)

BFS explores all neighbors at the current depth before moving deeper. It uses:

- A **queue**
- Level-wise traversal

BFS Pseudocode:

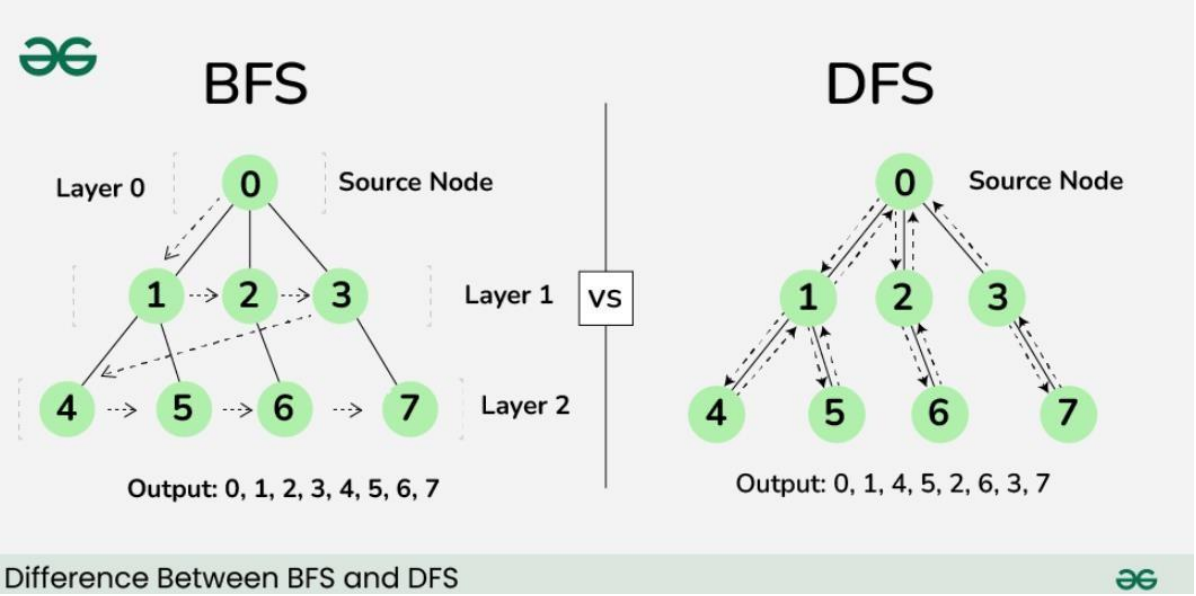
```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Diagram – DFS vs BFS Traversal

Here's a visual comparison of DFS and BFS traversal:



```
# Undirected Graph Representation
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

# DFS Recursive
def dfs(graph, vertex, visited=None):
    if visited is None:
        visited = set()
    visited.add(vertex)
    print(vertex, end=' ')
    for neighbor in graph[vertex]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

# BFS
from collections import deque
def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print(vertex, end=' ')
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# Run Traversals
print("DFS Traversal:")
dfs(graph, 'A')
print("\nBFS Traversal:")
bfs(graph, 'A')
```

Applications

- **Social Networks**: Finding connections between users
- **Web Crawlers**: Navigating linked pages
- **Network Routing**: Finding shortest paths
- **AI Pathfinding**: Game development and robotics
- **Cycle Detection**: In dependency graphs or circuits

Conclusion

In this practical, we implemented DFS and BFS algorithms for an undirected graph. DFS was implemented recursively, while BFS used a queue. These traversal techniques are foundational for solving graph-based problems in computer science and engineering.

Experiment No.2

Aim

- To implement the A* search algorithm in Python for solving a pathfinding problem in a maze or grid-based environment.

Objectives

1. Understand the concept of informed (heuristic-based) search algorithms.
2. Learn the working of the A* algorithm for optimal pathfinding.
3. Implement A* for solving a search problem with real-world constraints.
4. Evaluate how heuristics influence the performance of the search.

Theory

Q *What is A* Search?*

A* is a widely-used **informed search algorithm** that finds the shortest path from a start node to a goal node using both:

- **g(n)**: Actual cost from the start to the current node
- **h(n)**: Heuristic estimate from the current node to the goal
- **f(n) = g(n) + h(n)**: Total estimated cost of the cheapest solution through node n

It combines the strengths of:

- **Uniform-cost search** (optimal but slow)
- **Greedy best-first search** (fast but not always optimal

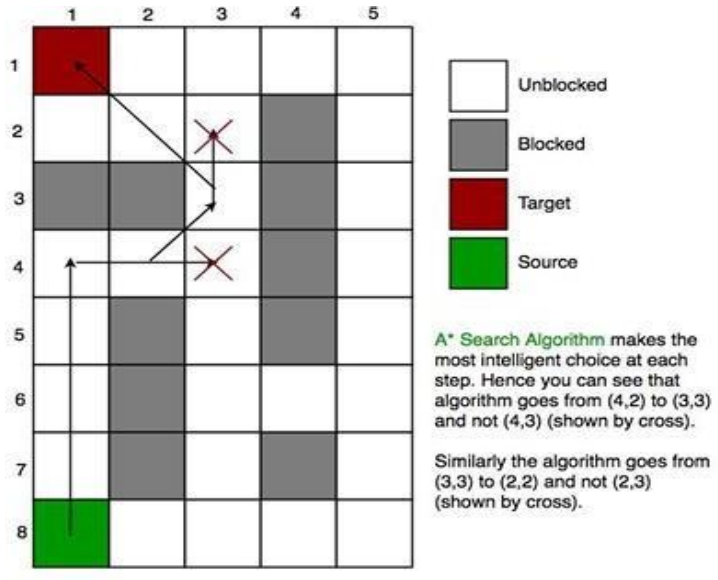


Figure 1: A* Search Algorithm.

A* Cost Function

The total cost function **f(n)** used in A* is:

f(n)=g(n)+h(n)f(n) = g(n) + h(n)

Where:

- **g(n)** = Actual cost from the start node to the current node **n**
- **h(n)** = Heuristic estimate of the cost from **n** to the goal
- **f(n)** = Total estimated cost of the cheapest solution through node **n**

Heuristic Functions

1Manhattan Distance

1. Euclidean Distance

\The heuristic must be **admissible** (never overestimates the true cost) to ensure A* returns the optimal path.

How A* Works

1. Initialize the **open list** with the start node.
2. Loop:
    - Pick the node with the **lowest f(n)** from the open list.
    - If it is the goal, stop.
    - Else, generate its neighbors and calculate their **f, g, and h**.
    - Add/update them in the open list.
3. Maintain a **closed list** to avoid revisiting nodes.

Code:

```
import heapq

# Define the grid: 0 = free path, 1 = wal
```

```
maze = [
    [0, 1, 0, 0, 0],
    [0, 1, 0, 1, 0],
   [0, 0, 0, 1, 0],
   [1, 1, 0, 1, 0],
   [0, 0, 0, 0, 0]
]
start = (0, 0)
goal = (4, 4)
# Heuristic function: Manhattan distance
def heuristic(a, b):
   return abs(a[0] - b[0]) + abs(a[1] - b[1])
# A* Algorithm
def astar(maze, start, goal):
   rows, cols = len(maze), len(maze[0])
   open_set = []
   heapq.heappush(open_set, (0 + heuristic(start, goal), 0, start, [start]))
   visited = set()
   while open_set:
      f, g, current, path = heapq.heappop(open_set)
      if current in visited:
         continue
      visited.add(current)
```

```
        if current == goal:
            return path
        for dx, dy in [(-1,0), (1,0), (0,-1), (0,1)]:
            x, y = current[0] + dx, current[1] + dy
  next_node = (x, y)
            if 0 <= x < rows and 0 <= y < cols and maze[x][y] == 0:
                if next_node not in visited:
                    new_g = g + 1
                    new_f = new_g + heuristic(next_node, goal)
                    heapq.heappush(open_set, (new_f, new_g, next_node, path +
[next_node]))
    return None
# Run the algorithm
path = astar(maze, start, goal)
print("A* Path:", path)
```

Application: Grid Pathfinding

- Environment is modeled as a **2D grid**.
- Cells are either **free (0)** or **blocked (1)**.
- **Start and goal cells** are specified.
- The algorithm must find the **shortest valid path** from start to goal.

Conclusion

In this practical, we implemented the *A* search algorithm to find an optimal path in a grid-based search space. By incorporating a heuristic (e.g., Manhattan distance), A* efficiently reduced the number of explored nodes while maintaining optimality.

Experiment No.3

Implement Alpha-Beta Tree search for any game search problem.

Aim

- To enhance the Minimax algorithm using Alpha-Beta Pruning to reduce computation time in decision-making for a two-player game.

Objectives

1. Understand adversarial search strategies in AI.
2. Implement the Minimax algorithm for two-player turn-based games.
3. Apply Alpha-Beta Pruning to optimize the search tree and reduce unnecessary evaluations.
4. Analyze how pruning improves performance without affecting the correctness of the solution.

Theory

Q *What is Minimax Algorithm?*

Minimax is a recursive algorithm used in decision-making and game theory for minimizing the possible loss in a worst-case scenario. In games like Tic Tac Toe or Chess:

- The **maximizing player (AI)** tries to get the highest score.
- The **minimizing player (Human)** tries to minimize the score.

Limitations of Minimax

- Explores all possible game states (branches), which becomes computationally expensive as depth increases.

Alpha-Beta Pruning

- Improves efficiency by pruning branches that don't affect the final decision.
- Reduces computation time without affecting correctness.

How It Works in Tic Tac Toe

- The board is represented as a list of 9 elements.
- The AI ('X') maximizes its score, while the human ('O') minimizes it.
- Game ends when either player wins or the board is full.
- Alpha-Beta pruning cuts off unnecessary branches during recursion, speeding up decision-making.

Code:

- `evaluate(board)` : Assigns a score
- `minimax_with_pruning(board, depth, is_max, alpha, beta)` : Core logic with pruning
- `find_best_move(board)` : Invokes minimax on all valid options

```
import math

# Initialize board

board = [[' ' for _ in range(3)] for _ in range(3)]

# Print current board state

def print_board(b):
```

```
for row in b:
            print('|'.join(row))
            print('-' * 5)
        # Check for any moves left
        def is_moves_left(b):
            return any(cell == ' ' for row in b for cell in row)
        # Evaluate board state
        def evaluate(b):
            for i in range(3):
                if b[i][0] == b[i][1] == b[i][2] != ' ':
                    return 10 if b[i][0] == 'X' else -10
                if b[0][i] == b[1][i] == b[2][i] != ' ':
                    return 10 if b[0][i] == 'X' else -10
            if b[0][0] == b[1][1] == b[2][2] != ' ':
                return 10 if b[0][0] == 'X' else -10
            if b[0][2] == b[1][1] == b[2][0] != ' ':
                return 10 if b[0][2] == 'X' else -10
            return 0
        # Minimax with Alpha-Beta Pruning
        def minimax(b, depth, is_max, alpha, beta):
            score = evaluate(b)
            # Terminal condition
            if score != 0 or not is_moves_left(b):
```

```
return score
        if is_max:  # Maximizing player (AI = X)
            max_eval = -math.inf
            for i in range(3):
                for j in range(3):
                    if b[i][j] == ' ':
                        b[i][j] = 'X'
                        eval_val = minimax(b, depth + 1, False, alpha, beta)
                        b[i][j] = ' '
                        max_eval = max(max_eval, eval_val)
                        alpha = max(alpha, eval_val)
                        if beta <= alpha:
                            break
            return max_eval
        else:  # Minimizing player (Human = O)
            min_eval = math.inf
            for i in range(3):
                for j in range(3):
                    if b[i][j] == ' ':
                        b[i][j] = 'O'
                        eval_val = minimax(b, depth + 1, True, alpha, beta)
                        b[i][j] = ' '
                        min_eval = min(min_eval, eval_val)
```

```
beta = min(beta, eval_val)
        if beta <= alpha:
                        break
            return min_eval
        # Best move selector for AI
        def find_best_move(b):
            best_val = -math.inf
            best_move = (-1, -1)
            for i in range(3):
                for j in range(3):
                    if b[i][j] == ' ':
                        b[i][j] = 'X'
                        move_val = minimax(b, 0, False, -math.inf, math.inf)
                        b[i][j] = ' '
                        if move_val > best_val:
                            best_val = move_val
                            best_move = (i, j)
            return best_move
        # Play against AI
        def play():
            while True:
                print_board(board)
                if evaluate(board) != 0 or not is_moves_left(board):
```

```
break
# User turn
        try:
            row = int(input("Enter your move row (0–2): "))
            col = int(input("Enter your move col (0–2): "))
        except ValueError:
            print("Invalid input! Enter numbers 0, 1, or 2.")
            continue
        if 0 <= row < 3 and 0 <= col < 3 and board[row][col] == ' ':
            board[row][col] = 'O'
        else:
            print("Invalid move. Try again.")
            continue
     # AI turn
        if is_moves_left(board) and evaluate(board) == 0:
            ai_move = find_best_move(board)
            board[ai_move[0]][ai_move[1]] = 'X'
    # Game Over
    print_board(board)
    final_score = evaluate(board)
    if final_score == 10
```

```
            print("AI Wins!")
        elif final_score == -10:
            print("You Win!")
        else:
            print("Draw!")
    # Run the game
    play()
```

Conclusion

In this practical, we implemented the Minimax algorithm and optimized it using Alpha-Beta Pruning for Tic Tac Toe. The pruning reduced the number of states evaluated, improving efficiency without affecting decision accuracy.

Experiment No.4

Aim

To implement a solution for a Constraint Satisfaction Problem (CSP) using **Backtracking** and **Branch and Bound** techniques for the **N-Queens problem**, ensuring all constraints are satisfied.

Objectives

1. Understand the concept of Constraint Satisfaction Problems (CSPs).
2. Apply backtracking and branch-and-bound techniques to solve CSPs.
3. Implement the N-Queens problem using recursive search.
4. Analyze the efficiency of constraint-based search algorithms.
5. Explore real-world applications of CSPs in AI and optimization.

Theory

What is a Constraint Satisfaction Problem (CSP)?

A CSP consists of:

- **Variables**: Elements to be assigned (e.g., positions of queens).
- **Domains**: Possible values for each variable.
- **Constraints**: Rules that restrict combinations of values (e.g., no two queens attacking each other).

N-Queens Problem

Place **N queens** on an **N×N chessboard** such that:

- No two queens share the same row, column, or diagonal.

Backtracking

Backtracking is a **depth-first search** technique:

- Try placing a queen row by row.
- If a conflict arises, **backtrack** and try a different position.

Branch and Bound

Branch and Bound improves backtracking by:

- **Pruning** branches that violate constraints early.
- Avoids exploring paths that cannot lead to a solution

Diagram – N-Queens (Example for N=4) Solution:

. Q . .

. . . Q

Q . . .

. . Q .

Each row has one queen, and no two queens attack each other.

Python Code – N-Queens Using Backtracking

```
def is_safe(board, row, col, n):
    # Check column
    for i in range(row):
        if board[i][col] == 1:
            return False
    # Check left diagonal
    for i, j in zip(range(row-1, -1, -1), range(col-1, -1, -1)):
        if board[i][j] == 1:
            return False
    # Check right diagonal
    for i, j in zip(range(row-1, -1, -1), range(col+1, n)):
        if board[i][j] == 1:
            return False
    return True
```

```
print_board(board) final_score =

evaluate(board) if final_score == 10:

        print("AI Wins!")

    elif final_score == -10:

        print("You Win!")

else:

        print("It's a Draw!")

# Uncomment to play:

# play_game()
```

Conclusion

In this practical, we implemented the **Minimax algorithm** to build an AI for the game of **Tic Tac Toe**. The algorithm correctly simulates all future moves and helps the AI choose the best possible move assuming the opponent also plays optimally.

```
def solve_n_queens(board, row, n): if row
== n:
                print_board(board)
                return True
            for col in range(n):
                if is_safe(board, row, col, n):
                    board[row][col] = 1
                    if solve_n_queens(board, row + 1, n):
                        return True
                    board[row][col] = 0  # Backtrack
            return False
        def print_board(board):
            for row in board:
                print(" ".join("Q" if col else "." for col in row))
            print()
        # Driver code
        n = 4
        board = [[0]*n for _ in range(n)]
        solve_n_queens(board, 0, n)
```

Applications

- **Scheduling Problems**: Assigning tasks without conflict
- **Sudoku Solver**: Filling cells with constraints
- **Map Coloring**: Assigning colors without adjacent matches
- **Resource Allocation**: Assigning limited resources optimally
- **AI Planning**: Robotics, game strategies, and decision-making

Conclusion

In this practical, we solved the N-Queens problem using **Backtracking** and **Branch and Bound** techniques. These methods efficiently explore the solution space while respecting constraints, making them powerful tools for solving CSPs in AI and optimization.

Experiment Number 5

Aim

To implement the **Greedy Search Algorithm** for solving the **Selection Sort** problem and understand how greedy decision-making leads to efficient sorting.

Objectives

1. Understand the concept of greedy algorithms and their characteristics.
2. Apply greedy logic to implement selection sort.
3. Analyze the time and space complexity of selection sort.
4. Explore real-world applications of greedy sorting techniques.
5. Compare greedy sorting with other sorting algorithms.

Theory

What is a Greedy Algorithm?

A greedy algorithm makes **locally optimal choices** at each step with the hope of finding a **global optimum**. It does not backtrack or reconsider decisions.

Selection Sort as a Greedy Algorithm

Selection Sort follows a greedy strategy:

- At each iteration, **select the minimum element** from the unsorted part.
- Swap it with the first unsorted element.
- Repeat until the array is sorted.

Greedy Choice: Always pick the smallest remaining element.

Diagram – Selection Sort (Example)

Unsorted Array: [64, 25, 12, 22, 11]

Pass 1: Select 11 → Swap with 64 → [11, 25, 12, 22, 64]

Pass 2: Select 12 → Swap with 25 → [11, 12, 25, 22, 64]

Pass 3: Select 22 → Swap with 25 → [11, 12, 22, 25, 64]

Pass 4: Already sorted → [11, 12, 22, 25, 64

Python Code – Selection Sort

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        # Find the minimum element in remaining
unsorted array
        for j in range(i+1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        # Swap the found minimum with the first
unsorted element
        arr[i], arr[min_index] = arr[min_index], arr[i]
# Example usage
arr = [64, 25, 12, 22, 11]
print("Original array:", arr)
selection_sort(arr)
print("Sorted array:", arr)
```

Applications

- **Small datasets** where simplicity is preferred over speed
- **Teaching sorting logic** in academic settings
- **Greedy-based decision systems** like tournament ranking
- **Embedded systems** with limited memory and processing power

Conclusion

In this practical, we implemented **Selection Sort** using a **Greedy Search Algorithm**. The algorithm consistently selects the minimum element, demonstrating greedy decision-making. Though not the most efficient for large datasets, it is conceptually simple and useful for understanding greedy strategies.

Experiment Number 6

Aim

To design and implement an elementary chatbot for a customer interaction application using basic rule-based or keyword-matching logic.

Objectives

1. Understand the concept and architecture of chatbots.
2. Develop a simple chatbot using conditional logic or pattern matching.
3. Simulate customer interaction scenarios such as FAQs or service queries.
4. Explore the limitations and scope of elementary chatbots.
5. Analyze how chatbots improve customer service automation.

Theory

What is a Chatbot?

A **chatbot** is a software application designed to simulate human conversation. It interacts with users via text or voice, often used in customer service, e-commerce, and support systems.

Types of Chatbots

- **Rule-Based Chatbots**: Use predefined rules and keywords to respond.
- **AI-Based Chatbots**: Use NLP and machine learning for dynamic responses.

Elementary Chatbot Logic

This practical focuses on a **rule-based chatbot** using:

- **If-else conditions**
- **Keyword matching**
- **Simple response templates**

Diagram – Chatbot Flow

User Input → Keyword Matching → Predefined Response → Display Output

Example:

User: "What are your working hours?"

Bot: "We are open from 9 AM to 6 PM, Monday to Saturday."

Python Code – Rule-Based Chatbot

```
def chatbot():
    print("Welcome to QuickHelp Customer Support!")
    print("Type 'exit' to end the chat.\n")
    while True:
        user_input = input("You: ").lower()
        if "hello" in user_input or "hi" in user_input:
            print("Bot: Hello! How can I assist you today?")
        elif "hours" in user_input or "timing" in user_input:
            print("Bot: We are open from 9 AM to 6 PM, Monday to Saturday.")
        elif "price" in user_input or "cost" in user_input:
            print("Bot: Our pricing starts at ₹499. Would you like a detailed quote?")
        elif "location" in user_input or "address" in user_input:
            print("Bot: We are located at MG Road, Pune.")
        elif "exit" in user_input:
            print("Bot: Thank you for chatting with us. Have a great day!")
            break
        else:
            print("Bot: I'm sorry, I didn't understand that. Could you please rephrase?")
# Run the chatbot
chatbot()
```

Applications

- **Customer Support**: Answering FAQs and guiding users
- **E-commerce**: Product queries and order tracking
- **Healthcare**: Appointment booking and symptom triage
- **Education**: Student helpdesk and course info
- **Banking**: Account queries and service navigation

Conclusion

In this practical, we developed a basic rule-based chatbot for customer interaction. The chatbot uses keyword matching to simulate conversation and respond to common queries. While limited in scope, it demonstrates the foundational logic behind conversational systems.

Experiment Number 7

Aim

To design and implement an elementary expert system for hospitals and medical facilities that assists in basic diagnosis or patient triage using rule-based logic.

Objectives

1. Understand the concept and architecture of expert systems.
2. Develop a rule-based expert system for medical decision support.
3. Simulate basic diagnostic or advisory interactions.
4. Explore the role of inference engines and knowledge bases.
5. Analyze the limitations and scope of expert systems in healthcare.

Theory

What is an Expert System?

An **expert system** is a computer program that simulates the decision-making ability of a human expert. It uses:

- **Knowledge Base**: A set of rules and facts.
- **Inference Engine**: Applies rules to known facts to deduce new facts.
- **User Interface**: Allows interaction with users.

Application in Hospitals

Expert systems in healthcare can:

- Assist in **preliminary diagnosis**
- Recommend **tests or treatments**
- Help in **triage** (deciding urgency)
- Provide **drug interaction alerts**